Hoare Logic II

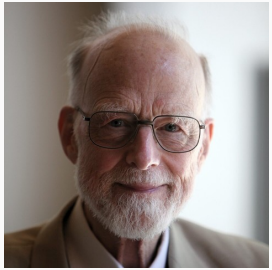
Invariants and weakest preconditions

Cristina David

Lecture 9

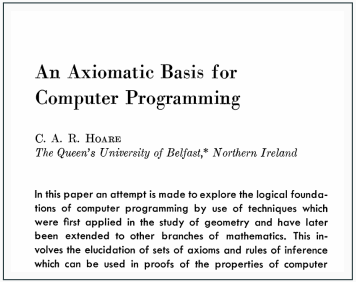
COMSM0067 — Advanced Topics in Programming Languages
University of Bristol

Where Hoare logic comes from



Sir Tony Hoare

Photo: Rama, CC BY-SA 2.0 fr



An Axiomatic Basis for Computer Programming

C. A. R. Hoare
The Queen's University of Belfast, Northern Ireland*

In this paper an attempt is made to explore the logical foundations of computer programming by use of techniques which were first applied in the study of geometry and have later been extended to other branches of mathematics. This involves the elucidation of sets of axioms and rules of inference which can be used in proofs of the properties of computer

Comm. ACM 12(10), 1969

- 1967: Floyd, *Assigning Meanings to Programs* — assertions on flowcharts.
- 1969: Hoare, *An Axiomatic Basis for Computer Programming* — the same idea as axioms and rules over program text. He wrote $P\,\{c\}\,R$; the braces moved later.
- 1975: Dijkstra, *Guarded Commands* — weakest preconditions, today's topic.
- Hoare also gave us Quicksort and CSP; Turing Award 1980.

Last time

- A third semantics: the Hoare triple $\{P\}\ c\ \{Q\}$, and partial correctness.
- Six rules for deriving triples.
- The assignment axiom runs backwards, so the swap proof went from right to left.

Last time

- A third semantics: the Hoare triple $\{P\}\ c\ \{Q\}$, and partial correctness.
- Six rules for deriving triples.
- The assignment axiom runs backwards, so the swap proof went from right to left.

And we ended on one unanswered question.

1. Where does the invariant come from?

Loop invariants

The rule that doesn't tell you how

$$\frac{\vdash \{I \wedge b\}\ c\ \{I\}}{\vdash \{I\}\ \text{while}\ b\ \text{do}\ c\ \{I \wedge \neg b\}}\ \text{While}$$

Everything hinges on I. But the rule is not syntax-directed in I: it does not compute the invariant, it only checks one you supply.

The rule that doesn't tell you how

$$\frac{\vdash \{I \wedge b\}\ c\ \{I\}}{\vdash \{I\}\ \text{while}\ b\ \text{do}\ c\ \{I \wedge \neg b\}}\ \text{While}$$

Everything hinges on I. But the rule is not syntax-directed in I: it does not compute the invariant, it only checks one you supply.

Finding I is the creative act of program verification.
It is the one step no calculus performs for you.

What an invariant must do

To prove $\{P\}$ while b do c $\{Q\}$ with invariant I, three obligations:

1. Establish: $P \Rightarrow I$ (I holds on entry)
2. Preserve: $\{I \wedge b\}\ c\ \{I\}$ (the body maintains it)
3. Conclude: $I \wedge \neg b \Rightarrow Q$ (on exit, I gives you Q)

What an invariant must do

To prove $\{P\}$ while b do c $\{Q\}$ with invariant I, three obligations:

1. Establish: $P \Rightarrow I$ (I holds on entry)
2. Preserve: $\{I \wedge b\}\ c\ \{I\}$ (the body maintains it)
3. Conclude: $I \wedge \neg b \Rightarrow Q$ (on exit, I gives you Q)

An invariant is a generalisation of the postcondition that is also inductive. Both halves matter: strong enough to give Q at the end, stable enough to survive a lap.

Diagnosing a candidate invariant

Given a loop and a candidate I, it is always exactly one of:

1. Not established. P doesn't give I. (I too strong at entry.)
2. Not preserved. The body can falsify it. (I too strong, or just wrong.)
3. Preserved but too weak. Survives every lap, but $I \wedge \neg b$ does not give Q.
4. Adequate. Preserved, and strong enough on exit.

Weakest preconditions

Mechanising the backward calculation

Last time, the swap proof ran backwards, and each assertion was computed, not guessed. Let us name the thing being computed.

Weakest precondition

$\mathrm{wp}(c, Q)$ is the weakest assertion P such that $\{P\}\ c\ \{Q\}$ holds.

Mechanising the backward calculation

Last time, the swap proof ran backwards, and each assertion was computed, not guessed. Let us name the thing being computed.

Weakest precondition

$\mathrm{wp}(c, Q)$ is the weakest assertion P such that $\{P\}\ c\ \{Q\}$ holds.

"Weakest" = most permissive = demands the least. Formally:

$$\vDash \{P\}\ c\ \{Q\} \qquad \text{iff} \qquad P \Rightarrow \mathrm{wp}(c, Q)$$

Mechanising the backward calculation

Last time, the swap proof ran backwards, and each assertion was computed, not guessed. Let us name the thing being computed.

Weakest precondition

$\mathrm{wp}(c, Q)$ is the weakest assertion P such that $\{P\}\ c\ \{Q\}$ holds.

"Weakest" = most permissive = demands the least. Formally:

$$\vDash \{P\}\ c\ \{Q\} \qquad \text{iff} \qquad P \Rightarrow \mathrm{wp}(c, Q)$$

So wp turns a verification question into an implication — the kind of thing you hand to a solver.

The wp calculus (loop-free)

$$
\begin{aligned}
\mathrm{wp}(\mathsf{skip}, Q) &= Q \\
\mathrm{wp}(x := a, Q) &= Q[a/x] \qquad \leftarrow \text{ the assignment axiom, as a function} \\
\mathrm{wp}(c_1; c_2, Q) &= \mathrm{wp}(c_1,\ \mathrm{wp}(c_2, Q)) \\
\mathrm{wp}(\mathsf{if}\ b\ \mathsf{then}\ c_1\ \mathsf{else}\ c_2, Q) &= (b \Rightarrow \mathrm{wp}(c_1, Q)) \wedge (\neg b \Rightarrow \mathrm{wp}(c_2, Q))
\end{aligned}
$$

The wp calculus (loop-free)

$$
\begin{aligned}
\mathrm{wp}(\mathsf{skip}, Q) &= Q \\
\mathrm{wp}(x := a, Q) &= Q[a/x] \qquad \leftarrow \text{ the assignment axiom, as a function} \\
\mathrm{wp}(c_1; c_2, Q) &= \mathrm{wp}(c_1,\ \mathrm{wp}(c_2, Q)) \\
\mathrm{wp}(\text{if } b \text{ then } c_1 \text{ else } c_2, Q) &= (b \Rightarrow \mathrm{wp}(c_1, Q)) \wedge (\neg b \Rightarrow \mathrm{wp}(c_2, Q))
\end{aligned}
$$

Purely syntax-directed: recurse on c, right to left, and the precondition falls out. No invention.

The six rules of Lecture 8, read as an algorithm.

But loops break the recursion

There is no clause for while.

wp(while b do c, Q) is a fixed point — it unfolds to

$$(\neg b \wedge Q) \ \vee \ (b \wedge \mathrm{wp}(c, \mathrm{wp}(\text{while} \ldots, Q))) \ \vee \ \cdots$$

an unbounded disjunction over the number of iterations. Not computable by simple recursion.

But loops break the recursion

There is no clause for while.

wp(while b do c, Q) is a fixed point — it unfolds to

$$(\neg b \wedge Q) \ \vee \ (b \wedge \mathrm{wp}(c, \mathrm{wp}(\text{while} \ldots, Q))) \ \vee \ \cdots$$

an unbounded disjunction over the number of iterations. Not computable by simple recursion.

This is why we annotate

The tool cannot find the loop's precondition by calculation. So you supply the invariant, and the tool checks it. The human does the creative step; the machine does the bookkeeping.

One law worth knowing

wp distributes over conjunction:

$$\mathrm{wp}(c,\ Q_1 \wedge Q_2) \ =\ \mathrm{wp}(c, Q_1) \wedge \mathrm{wp}(c, Q_2)$$

Useful: you can verify a conjunctive specification componentwise.

Verification conditions

To check $\{P\}\ c\ \{Q\}$: weave wp backwards through c, starting from Q. What is left is a purely logical formula,

$$P \Rightarrow \mathrm{wp}(c, Q)$$

a verification condition (VC). No program text left — just logic.

Verification conditions

To check $\{P\}\ c\ \{Q\}$: weave wp backwards through c, starting from Q. What is left is a purely logical formula,

$$P \;\Rightarrow\; \mathrm{wp}(c, Q)$$

a verification condition (VC). No program text left — just logic.

Example. $\{x > 0\}\ \ y := x + 1;\ z := y \cdot 2\ \ \{z > 2\}$

$$\mathrm{wp}(c,\ z > 2) \;=\; (x+1)\cdot 2 > 2 \qquad \rightsquigarrow \qquad \text{VC:}\quad x > 0 \;\Rightarrow\; (x+1)\cdot 2 > 2$$

Verification conditions

To check $\{P\}\ c\ \{Q\}$: weave wp backwards through c, starting from Q. What is left is a purely logical formula,

$$P \ \Rightarrow\ \mathrm{wp}(c, Q)$$

a verification condition (VC). No program text left — just logic.

Example. $\{x > 0\}\ \ y := x + 1;\ z := y \cdot 2\ \ \{z > 2\}$

$$\mathrm{wp}(c,\ z > 2)\ =\ (x+1)\cdot 2 > 2 \qquad \rightsquigarrow \qquad \text{VC:}\quad x > 0\ \Rightarrow\ (x+1)\cdot 2 > 2$$

VCs = what is left to prove once wp has been woven through the program.
A tool generates them; an solver proves them.

Worked example

Summing 0 to $n-1$

$$\{n \geq 0\} \;\; i := 0;\; s := 0;\; \text{while } i < n \text{ do } (s := s + i;\; i := i + 1) \;\; \{\; s = \textstyle\sum_{k=0}^{n-1} k \;\}$$

Summing 0 to $n-1$

$$\{n \geq 0\}\ \ i := 0;\ s := 0;\ \text{while } i < n \text{ do } (s := s + i;\ i := i + 1)\ \ \{\ s = \textstyle\sum_{k=0}^{n-1} k\ \}$$

First, the invariant. Try the postcondition itself: $s = \sum_{k=0}^{n-1} k$. Not preserved — it mentions n, not the running counter.

Summing 0 to $n-1$

$$\{n \geq 0\}\ \ i := 0;\ s := 0;\ \text{while } i < n \text{ do } (s := s + i;\ i := i + 1)\ \ \{\ s = \textstyle\sum_{k=0}^{n-1} k\ \}$$

First, the invariant. Try the postcondition itself: $s = \sum_{k=0}^{n-1} k$. Not preserved — it mentions n, not the running counter.

The fix: generalise n to i, and say where i lives.

$$I \ \equiv\ s = \sum_{k=0}^{i-1} k \ \wedge\ 0 \leq i \ \wedge\ i \leq n$$

The postcondition is the special case $i = n$.

Summing 0 to $n-1$

$$\{n \geq 0\}\ \ i := 0;\ s := 0;\ \text{while } i < n \text{ do } (s := s + i;\ i := i + 1)\ \ \{\ s = \textstyle\sum_{k=0}^{n-1} k\ \}$$

First, the invariant. Try the postcondition itself: $s = \sum_{k=0}^{n-1} k$. Not preserved — it mentions n, not the running counter.

The fix: generalise n to i, and say where i lives.

$$I \ \equiv\ s = \sum_{k=0}^{i-1} k \ \wedge\ 0 \leq i \ \wedge\ i \leq n$$

The postcondition is the special case $i = n$.

Then, no more invention: weave wp backwards, collect the VCs, discharge them.

The plan: a proof outline

$\{n \geq 0\}$ precondition P

$i := 0;$

$s := 0;$
$\{I\}$ annotation
while $i < n$ do

  $s := s + i;$

  $i := i + 1$

$\{Q\}$ postcondition Q

Given: P and Q from the spec, I from the annotation.

The plan: a proof outline

$\{n \geq 0\}$ precondition P

$i := 0;$

$s := 0;$
$\{I\}$ annotation
while $i < n$ do
  $\{I \wedge i < n\}$ While

  $s := s + i;$

  $i := i + 1$
  $\{I\}$ While
$\{I \wedge i \geq n\}$ While
$\{Q\}$ postcondition Q

Given: P and Q from the spec, I from the annotation.

From While: $I \wedge b$ entering the body, I leaving it, $I \wedge \neg b$ after the loop.

The plan: a proof outline

$\{n \geq 0\}$	precondition P
$i := 0;$	
$s := 0;$	
$\{I\}$	annotation
while $i < n$ do	
$\{I \wedge i < n\}$	While
$s := s + i;$	
$\{B_1\}$	$B_1 = \mathrm{wp}(i := i + 1,\ I)$
$i := i + 1$	
$\{I\}$	While
$\{I \wedge i \geq n\}$	While
$\{Q\}$	postcondition Q

Given: P and Q from the spec, I from the annotation.

From While: $I \wedge b$ entering the body, I leaving it, $I \wedge \neg b$ after the loop.

Computed by wp: bottom up, each from the assertion just below it.

The plan: a proof outline

$\{n \geq 0\}$	precondition P
$i := 0;$	
$s := 0;$	
$\{I\}$	annotation
while $i < n$ do	
$\{I \wedge i < n\}$	While
$\{B_0\}$	$B_0 = \mathsf{wp}(s := s + i,\ B_1)$
$s := s + i;$	
$\{B_1\}$	$B_1 = \mathsf{wp}(i := i + 1,\ I)$
$i := i + 1$	
$\{I\}$	While
$\{I \wedge i \geq n\}$	While
$\{Q\}$	postcondition Q

Given: P and Q from the spec, I from the annotation.

From While: $I \wedge b$ entering the body, I leaving it, $I \wedge \neg b$ after the loop.

Computed by wp: bottom up, each from the assertion just below it.

The plan: a proof outline

$\{n \geq 0\}$	precondition P
$i := 0;$	
$\{W_1\}$	$W_1 = \mathrm{wp}(s := 0,\ I)$
$s := 0;$	
$\{I\}$	annotation
while $i < n$ do	
$\{I \wedge i < n\}$	While
$\{B_0\}$	$B_0 = \mathrm{wp}(s := s + i,\ B_1)$
$s := s + i;$	
$\{B_1\}$	$B_1 = \mathrm{wp}(i := i + 1,\ I)$
$i := i + 1$	
$\{I\}$	While
$\{I \wedge i \geq n\}$	While
$\{Q\}$	postcondition Q

Given: P and Q from the spec, I from the annotation.

From While: $I \wedge b$ entering the body, I leaving it, $I \wedge \neg b$ after the loop.

Computed by wp: bottom up, each from the assertion just below it.

The plan: a proof outline

$\{n \geq 0\}$	precondition P
$\{W_0\}$	$W_0 = \mathsf{wp}(i := 0,\ W_1)$
$i := 0;$	
$\{W_1\}$	$W_1 = \mathsf{wp}(s := 0,\ I)$
$s := 0;$	
$\{I\}$	annotation
while $i < n$ do	
$\quad \{I \wedge i < n\}$	While
$\quad \{B_0\}$	$B_0 = \mathsf{wp}(s := s + i,\ B_1)$
$\quad s := s + i;$	
$\quad \{B_1\}$	$B_1 = \mathsf{wp}(i := i + 1,\ I)$
$\quad i := i + 1$	
$\quad \{I\}$	While
$\{I \wedge i \geq n\}$	While
$\{Q\}$	postcondition Q

Given: P and Q from the spec, I from the annotation.

From While: $I \wedge b$ entering the body, I leaving it, $I \wedge \neg b$ after the loop.

Computed by wp: bottom up, each from the assertion just below it.

The plan: a proof outline

$\{n \geq 0\}$	precondition P
$\{W_0\}$ (1)	$W_0 = \mathrm{wp}(i := 0,\ W_1)$
$i := 0;$	
$\{W_1\}$	$W_1 = \mathrm{wp}(s := 0,\ I)$
$s := 0;$	
$\{I\}$	annotation
while $i < n$ do	
$\{I \wedge i < n\}$	While
$\{B_0\}$ (2)	$B_0 = \mathrm{wp}(s := s + i,\ B_1)$
$s := s + i;$	
$\{B_1\}$	$B_1 = \mathrm{wp}(i := i + 1,\ I)$
$i := i + 1$	
$\{I\}$	While
$\{I \wedge i \geq n\}$	While
$\{Q\}$ (3)	postcondition Q

Given: P and Q from the spec, I from the annotation.

From While: $I \wedge b$ entering the body, I leaving it, $I \wedge \neg b$ after the loop.

Computed by wp: bottom up, each from the assertion just below it.

Wherever two assertions stack, the upper must imply the lower — a Conseq step. Those are the VCs:

(1) establish $n \geq 0 \Rightarrow W_0$

(2) preserve $I \wedge i < n \Rightarrow B_0$

(3) exit $I \wedge i \geq n \Rightarrow Q$

Step 1 — establish: wp through the initialisation

Start from I, just before the loop, and push it up through $s := 0$, then $i := 0$.

Step 1 — establish: wp through the initialisation

Start from I, just before the loop, and push it up through $s := 0$, then $i := 0$.

Before $s := 0$: $\quad W_1 = \mathsf{wp}(s := 0,\ I) = I[0/s]$

$$W_1 \ = \ 0 = \sum_{k=0}^{i-1} k \ \wedge \ 0 \leq i \ \wedge \ i \leq n$$

Step 1 — establish: wp through the initialisation

Start from I, just before the loop, and push it up through $s := 0$, then $i := 0$.

Before $s := 0$: $W_1 = \mathrm{wp}(s := 0,\ I) = I[0/s]$

$$W_1 \ = \ 0 = \sum_{k=0}^{i-1} k \ \wedge \ 0 \leq i \ \wedge \ i \leq n$$

Before $i := 0$: $W_0 = \mathrm{wp}(i := 0,\ W_1) = W_1[0/i]$

$$W_0 \ = \ 0 = \sum_{k=0}^{-1} k \ \wedge \ 0 \leq 0 \ \wedge \ 0 \leq n$$

Step 1 — establish: wp through the initialisation

Start from I, just before the loop, and push it up through $s := 0$, then $i := 0$.

Before $s := 0$: $\quad W_1 = \mathrm{wp}(s := 0,\ I) = I[0/s]$

$$W_1 \ = \ 0 = \sum_{k=0}^{i-1} k \ \wedge \ 0 \leq i \ \wedge \ i \leq n$$

Before $i := 0$: $\quad W_0 = \mathrm{wp}(i := 0,\ W_1) = W_1[0/i]$

$$W_0 \ = \ 0 = \sum_{k=0}^{-1} k \ \wedge \ 0 \leq 0 \ \wedge \ 0 \leq n$$

VC (1): $\quad n \geq 0 \ \Rightarrow \ 0 = \sum_{k=0}^{-1} k \wedge 0 \leq 0 \wedge 0 \leq n$

Step 1 — establish: wp through the initialisation

Start from I, just before the loop, and push it up through $s := 0$, then $i := 0$.

Before $s := 0$: $\quad W_1 = \mathrm{wp}(s := 0,\ I) = I[0/s]$

$$W_1 \ = \ 0 = \sum_{k=0}^{i-1} k \ \wedge \ 0 \le i \ \wedge \ i \le n$$

Before $i := 0$: $\quad W_0 = \mathrm{wp}(i := 0,\ W_1) = W_1[0/i]$

$$W_0 \ = \ 0 = \sum_{k=0}^{-1} k \ \wedge \ 0 \le 0 \ \wedge \ 0 \le n$$

VC (1): $\quad n \ge 0 \ \Rightarrow \ 0 = \sum_{k=0}^{-1} k \wedge 0 \le 0 \wedge 0 \le n$

Discharge: the empty sum is 0; $0 \le 0$; and $n \ge 0$ gives $0 \le n$. ✓

Step 2 — preserve: wp through the body

Start from I, at the end of the body, and push it up through $i := i + 1$, then $s := s + i$.

Step 2 — preserve: wp through the body

Start from I, at the end of the body, and push it up through $i := i + 1$, then $s := s + i$.

Before $i := i + 1$: $\quad B_1 = \mathsf{wp}(i := i + 1,\ I) = I[i + 1/i]$

$$B_1 \;=\; s = \sum_{k=0}^{(i+1)-1} k \wedge 0 \le i + 1 \wedge i + 1 \le n \;\equiv\; s = \sum_{k=0}^{i} k \wedge -1 \le i \wedge i < n$$

Step 2 — preserve: wp through the body

Start from I, at the end of the body, and push it up through $i := i+1$, then $s := s+i$.

Before $i := i+1$: $\quad B_1 = \mathrm{wp}(i := i+1,\ I) = I[i+1/i]$

$$B_1 \ = \ s = \sum_{k=0}^{(i+1)-1} k \wedge 0 \le i+1 \wedge i+1 \le n \ \equiv \ s = \sum_{k=0}^{i} k \wedge -1 \le i \wedge i < n$$

Before $s := s+i$: $\quad B_0 = \mathrm{wp}(s := s+i,\ B_1) = B_1[s+i/s]$

$$B_0 \ = \ s+i = \sum_{k=0}^{i} k \ \wedge \ -1 \le i \ \wedge \ i < n$$

Step 2 — preserve: the VC

VC (2): $I \wedge i < n \Rightarrow B_0$, i.e.

$$s = \sum_{k=0}^{i-1} k \wedge 0 \leq i \leq n \wedge i < n \;\Rightarrow\; s + i = \sum_{k=0}^{i} k \;\wedge\; -1 \leq i \;\wedge\; i < n$$

Step 2 — preserve: the VC

VC (2): $I \wedge i < n \;\Rightarrow\; B_0$, i.e.

$$s = \sum_{k=0}^{i-1} k \wedge 0 \le i \le n \wedge i < n \;\Rightarrow\; s + i = \sum_{k=0}^{i} k \;\wedge\; -1 \le i \;\wedge\; i < n$$

Discharge: since $i \ge 0$, split off the last term: $\sum_{k=0}^{i} k = \sum_{k=0}^{i-1} k + i = s + i$. Also $0 \le i$ gives $-1 \le i$, and $i < n$ is assumed. ✓

Step 2 — preserve: the VC

VC (2): $I \wedge i < n \;\Rightarrow\; B_0$, i.e.

$$s = \sum_{k=0}^{i-1} k \wedge 0 \le i \le n \wedge i < n \;\Rightarrow\; s + i = \sum_{k=0}^{i} k \;\wedge\; -1 \le i \;\wedge\; i < n$$

Discharge: since $i \ge 0$, split off the last term: $\sum_{k=0}^{i} k = \sum_{k=0}^{i-1} k + i = s + i$. Also $0 \le i$ gives $-1 \le i$, and $i < n$ is assumed. ✓

Why $0 \le i$?

Drop it and VC (2) fails: $i = -1,\ s = 0$ satisfies the rest of I and $i < n$, yet $s + i = -1 \neq 0 = \sum_{k=0}^{-1} k$. Splitting off the last term needs $i \ge 0$.

Step 3 — exit: nothing to compute

The loop is the last command, so the assertion after it is Q itself. No wp step is needed.

Step 3 — exit: nothing to compute

The loop is the last command, so the assertion after it is Q itself. No wp step is needed.

VC (3): $I \wedge i \geq n \Rightarrow Q$, i.e.

$$s = \sum_{k=0}^{i-1} k \ \wedge \ 0 \leq i \leq n \ \wedge \ i \geq n \ \Rightarrow \ s = \sum_{k=0}^{n-1} k$$

Step 3 — exit: nothing to compute

The loop is the last command, so the assertion after it is Q itself. No wp step is needed.

VC (3): $I \wedge i \geq n \Rightarrow Q$, i.e.

$$s = \sum_{k=0}^{i-1} k \ \wedge \ 0 \leq i \leq n \ \wedge \ i \geq n \ \Rightarrow \ s = \sum_{k=0}^{n-1} k$$

Discharge: $i \leq n$ and $i \geq n$ force $i = n$; substitute. ✓

On a real tool: Dafny

The same program in Dafny

```
function Sum(m: int): int  // 0+1+...+(m-1)
{ if m <= 0 then 0 else Sum(m-1) + (m-1) }

method SumUpTo(n: int) returns (s: int)
  requires n >= 0                  // P
  ensures  s == Sum(n)             // Q
{
  var i := 0;  s := 0;
  while i < n
    invariant 0 <= i <= n          // I
    invariant s == Sum(i)
  {
    s := s + i;
    i := i + 1;
  }
}
```

- `requires`/`ensures`: the triple's P and Q.
- `invariant`: our I, one conjunct per line.
- No Σ in Dafny, so define it: $\mathrm{Sum}(i) = \sum_{k=0}^{i-1} k$.
- No proof is written. Steps 1–3 are the tool's job.

Running it

```
$ dafny verify sum.dfy

Dafny program verifier finished with 2 verified, 0 errors
```

Two declarations checked: `Sum` and `SumUpTo`.

Running it

```
$ dafny verify sum.dfy

Dafny program verifier finished with 2 verified, 0 errors
```

Two declarations checked: `Sum` and `SumUpTo`.

Under the hood, the same pipeline as on the board:

$$\text{annotated program} \xrightarrow{\text{wp-style VCgen (Boogie)}} \text{VCs} \xrightarrow{\text{SMT (Z3)}} \checkmark \text{ or a failing VC}$$

When it fails, it tells you which VC

Drop $i \leq n$ from I:

```
    invariant 0 <= i
```

```
sum.dfy(7,0): Error: a postcondition
  could not be proved on this return path
sum.dfy(6,11): Related location: this is
  the postcondition that could not be proved
```

Exit VC (3): $I \wedge i \geq n$ no longer forces $i = n$.

Break the body (off by one):

```
    s := s + i + 1;
```

```
sum.dfy(11,14): Error: this invariant
  could not be proved to be maintained
  by the loop
```

Preserve VC (2): $s + i + 1 \neq \sum_{k=0}^{i} k$.

Each error message is one of our three VCs, pointed at the source line it came from.

Beyond Dafny: Hoare-logic verifiers for real languages

Tool	Language	Developed at	Proves VCs with
Dafny	its own*	Microsoft, now AWS	Boogie → Z3
Verus	Rust	CMU, Microsoft, VMware	Z3
Prusti	Rust	ETH Zurich	Viper → Z3
Creusot	Rust	Inria (France)	Why3 → SMT
Frama-C (WP plugin)	C	CEA & Inria (France)	Why3 / Alt-Ergo
SPARK (GNATprove)	Ada	AdaCore	Why3 → SMT
KeY	Java	KIT, Darmstadt, Chalmers	own prover
Nagini, Gobra	Python, Go	ETH Zurich	Viper → Z3

*compiled to C#, Java, Go, Python, JavaScript

Beyond Dafny: Hoare-logic verifiers for real languages

Tool	Language	Developed at	Proves VCs with
Dafny	its own*	Microsoft, now AWS	Boogie → Z3
Verus	Rust	CMU, Microsoft, VMware	Z3
Prusti	Rust	ETH Zurich	Viper → Z3
Creusot	Rust	Inria (France)	Why3 → SMT
Frama-C (WP plugin)	C	CEA & Inria (France)	Why3 / Alt-Ergo
SPARK (GNATprove)	Ada	AdaCore	Why3 → SMT
KeY	Java	KIT, Darmstadt, Chalmers	own prover
Nagini, Gobra	Python, Go	ETH Zurich	Viper → Z3

*compiled to C#, Java, Go, Python, JavaScript

Same recipe in all of them: you write `requires` / `ensures` / `invariant`; the tool weaves wp and hands the VCs to a solver.

Industry is paying for this

- Amazon (AWS). The IAM authorization engine — about a billion requests per second — was rewritten in Dafny, proved correct, and deployed in 2024. It also got faster.
- Microsoft. Z3 and Boogie, the engines under most of the tools on the previous slide, come from Microsoft Research; so did Dafny, and Verus is co-developed there.
- NVIDIA. Since 2019, rewriting security-critical firmware from C into SPARK.
- Airbus. Verifies avionics code with Frama-C.

Industry is paying for this

- Amazon (AWS). The IAM authorization engine — about a billion requests per second — was rewritten in Dafny, proved correct, and deployed in 2024. It also got faster.
- Microsoft. Z3 and Boogie, the engines under most of the tools on the previous slide, come from Microsoft Research; so did Dafny, and Verus is co-developed there.
- NVIDIA. Since 2019, rewriting security-critical firmware from C into SPARK.
- Airbus. Verifies avionics code with Frama-C.

The ideas from today — invariants, wp, VCs, SMT — are what these teams use.

Wrap

Today

- The invariant is the one creative step — a generalisation of Q that is also inductive.
- Weakest preconditions make the backward calculation an algorithm — for loop-free code. Loops need an annotation, because wp(while) is a fixed point.
- VCs are what is left once wp is woven through an annotated program: generation is calculation, discharge is logic (solver). Dafny does both.

Next time

Everything so far assumed variables are independent boxes.

Introduce the heap (pointers, aliasing) and the assignment axiom breaks: after $[x] := a$, you cannot substitute, because you do not know what *else* points at that cell.

Next time

Everything so far assumed variables are independent boxes.

Introduce the heap (pointers, aliasing) and the assignment axiom breaks: after $[x] := a$, you cannot substitute, because you do not know what *else* points at that cell.

Separation logic (Lecture 10).

Questions?